

Reviewer Pack — Minimal Rank of Cocomplexes over Boolean Valuations

Entry 51fa0c1876f4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:27:09 UTC

Minimal Rank of Cocomplexes over Boolean Valuations

Entry ID: 51fa0c1876f4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 13:27:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Cocomplex Theory) **Field B** (complexity object): Communication Complexity (Boolean Valuations)

Statement:

For every Boolean valuation V on an n -bit string, the minimal rank of the cocomplex associated with V is upper bounded by the communication complexity of the disjointness problem for n bits, i.e., $E[\text{rank}(\text{cocomplex}(V))] \leq \text{CC_DISJ}(n)$.

Rationale (proposer's reasoning):

Cocomplex theory provides a geometric framework to study algebraic varieties and their properties. By mapping Boolean valuations to cocomplexes, we can leverage geometric tools to analyze communication complexity problems. This conjecture suggests that the rank of these cocomplexes is related to the difficulty of solving communication tasks.

Taxonomy category: Algebraic_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4a65133a6c7a1c17

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the average rank of the cocomplexes across all 30 seeds does not exceed the communication complexity of the disjointness problem for n bits by more than a threshold of 10%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "cocomplex theory" AND "communication complexity" AND boolean valuations" - "minimal rank" cocomplex AND communication complexity boolean valuation" - "disjointness problem" communication complexity AND cocomplex boolean valuation

Top relevant hits considered: - [http://arxiv.org/abs/1710.07343v1] Fractional Minimal Rank - [http://arxiv.org/abs/2208.01392v2] The Analytic Minimal Rank Sard Conjecture - [http://arxiv.org/abs/2406.15089v2] Sentences over Random Groups II: Sentences of Minimal Rank - [http://arxiv.org/abs/0908.4453v1] Depth-Independent Lower bounds on the Communication Complexity of Read-Once Boolean Formulas - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [s2:098989b418d361f352e4bf9] Towards Understanding the Min-Sum Message Passing Algorithm for the Minimum Weighted Vertex Cover Problem: An Analytical - [s2:b719e184c7a975eeae3a413b5e79b76d4ac3e530] Automata, Languages and Programming: 21st International Colloquium, Icalp 94 Jerusalem, Israel, July 11-14, 1994 : Proce

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def communication_complexity_disj(n):
        return n

    def construct_cocomplex(V):
        rank = len([v for v in V if sum(v) > 0])
        return rank

    def generate_boolean_valuation(n):
        return [random.randint(0, 1) for _ in range(n)]

    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Ensure at least 5 instances per size
            V = generate_boolean_valuation(n)
```

```

        rank = construct_cocomplex(V)
        total_rank += rank
        instances_tested += 1

    avg_rank = Fraction(total_rank, instances_tested)
    cc_disj_n = communication_complexity_disj(n)

    conjecture_holds = avg_rank <= Fraction(cc_disj_n * 1.10)
    counterexample = "" if conjecture_holds else f"rank={avg_rank}, expected={cc_disj_n}"

    return {
        "metric_name": "cocomplex_rank",
        "metric_value": float(avg_rank),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30)) # Default

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    avg_rank = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed64afa.py", line 62, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed64afa.py", line 38, in run_trial

```

    rank = construct_cocomplex(V)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ed64afa.py", line 25, in construct_cocomplex

```

    rank = len([v for v in V if sum(v) > 0])

```

^^^^^^
TypeError: 'int' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10343
2	preregistration	ollama_remote	glm4:latest	0	0	5635
3	novelty	ollama_remote	glm4:latest	0	0	4849
4	novelty	ollama_remote	glm4:latest	0	0	5976
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40653
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7975
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8329
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7191
9	judge	ollama_remote	glm4:latest	0	0	13893

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104844 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/51fa0c1876f4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/51fa0c1876f4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/51fa0c1876f4.tar.gz` (if generated)